TuitionPilot: Technical Documentation

FinTechathon 2026, International Track · Topic C: Cross-border & Student Finance Assistant
Team Borderless Scholars: Ekaterina Kulakova (lead), Dinara Khasyanova
Repository: https://github.com/Extravoganza/tuitionpilot · Version 0.3.2 (19 October 2026)

> **SIMULATION. No real money moves.** All universities, routes, quotes, payees and balances are demo fixtures. University rules are marked DEMO FIXTURE or NOT VERIFIED until confirmed with the official source.

1. Problem and scope

International students pay large, deadline-bound bills (tuition, rent, insurance) in a currency they do not earn. Four things go wrong again and again:

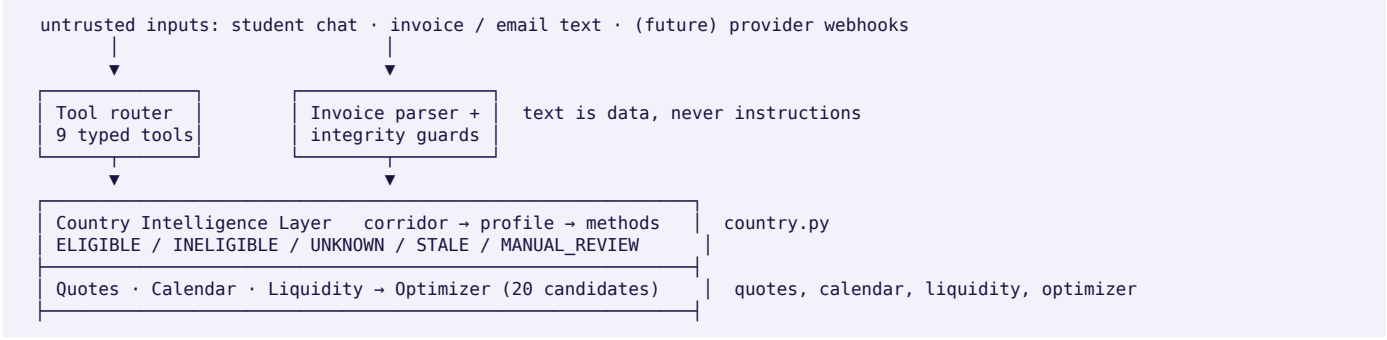
1. **The wrong method.** A university accepts some payment methods for some payers and not others. Students learn this after the money has left.
2. **Cost.** A bank wire hides an FX markup plus fixed and intermediary fees. Cards add about 3%.
3. **Time.** The deadline is the day the money must be *credited*, not sent. Weekends, holidays, cut-offs and university processing days eat the margin.
4. **Fraud.** "Our bank details have changed" emails target students, who pay institutions they have never visited.

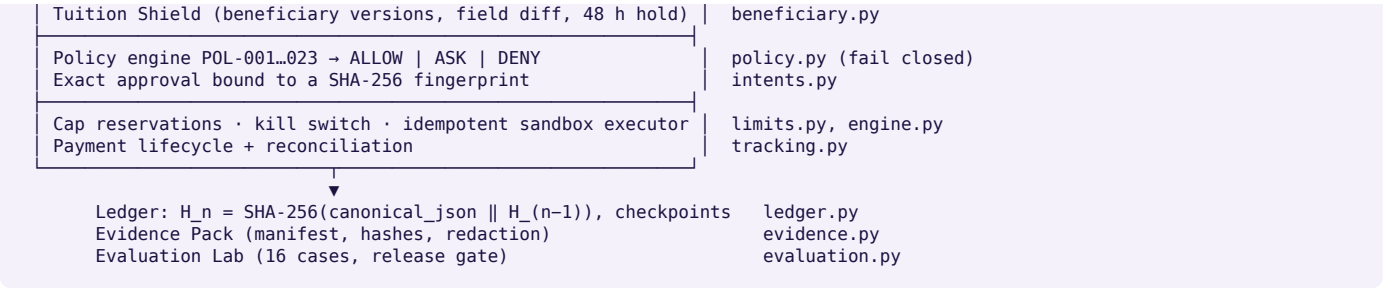
TuitionPilot is an AI-assisted payment agent for this one job. For a given university and payment direction it tells the student which methods are allowed and which documents are missing, compares the known all-in cost and arrival time of each route, picks the cheapest plan that passes every hard limit, checks the payee against a trusted version, asks for one exact approval, and then tracks the payment until the credit is matched.

China is the first country pack, with two separate corridors: paying a Chinese university from abroad (FROM_ABROAD_TO_CHINA) and paying a university abroad from China, for example by a parent (FROM_CHINA_TO_ABROAD). One finished, safe scenario is more useful than many shallow integrations.

In scope: country and corridor rules, method eligibility, deadline projection, route quotes, deadline-constrained optimizer, liquidity, Tuition Shield, policy engine, exact approvals, sandbox execution, payment lifecycle and reconciliation, ledger, evidence pack, evaluation lab, interactive website. **Out of scope:** real bank connections, KYC/AML, live market data, verified university data, production identity and key management (section 10).

2. Architecture





Package	Contents
`tuitionpilot/treasury/` (current core)	`currency`, `quotes`, `calendar`, `liquidity`, `optimizer`, `beneficiary`, `policy`, `intents`, `limits`, `engine`, `ledger`, `tools`, `stress`, `flagship`, `country`, `tracking`, `evidence`, `evaluation`, `china_demo`, `planner_api`
`tuitionpilot/` (v0.1 engine, kept)	Agent loop, permission tiers L0–L3, injection guards, FX tranche planner, backtest, LLM planner adapter, 8 sandbox scenarios
`demo/`	Static website (one HTML file + assets) with browser ports of the eligibility checker and the optimizer

About 5,700 lines of standard-library Python, **no runtime dependencies**. Money is stored in integer minor units with an explicit rate direction. Time is injected, so every run is reproducible with a seed.

3. Country Intelligence Layer (China-first)

Corridor resolver. `resolve_corridor(sender_country, university_country, payer_type)` returns FROM_ABROAD_TO_CHINA, FROM_CHINA_TO_ABROAD or NOT_SUPPORTED. An unsupported direction never falls back to a "close" corridor.

Profiles. A `CorridorProfile` holds the university, corridor, methods, rules, documents, calendar and sources. Profiles are versioned. Once a version has been used for a decision it is frozen: `replace_in_place` raises `ProfileFrozen`, and a change creates a new version, so old decisions can always be explained.

Sources. Each rule points to a `SourceRecord` with scope (university, corridor, payer types) and a validity date. A rule past its date is STALE; a source for another university or payer is a SOURCE_SCOPE_MISMATCH and cannot make a method eligible. The public pages of Tsinghua, Peking University, Xi'an Jiaotong and SAFE are listed only as NOT_VERIFIED research starting points.

Method eligibility. `check_method` is deterministic and returns one of:

Status	Meaning
ELIGIBLE	Every requirement met, sources fresh and in scope
INELIGIBLE	A hard requirement fails (wrong payer, currency, missing instrument)
UNKNOWN	No source says the method is accepted
STALE	The supporting rule is past its validity date
MANUAL_REVIEW	Needs a human check (for example, a scope mismatch)

`check_eligibility` combines methods and documents into a readiness status: READY, MISSING_DOCUMENTS, RULES_OUTDATED, NOT_SUPPORTED or MANUAL_REVIEW. Example from the fixtures: a parent paying from

China (corridor B) gets MISSING_DOCUMENTS until a relationship proof is added, then READY with B-WIRE.

Deadline projection. `project_deadline` works back from the due date: university processing buffer → provider arrival range in business days → cut-off and time zone → **last safe start date**.

4. Planning algorithms

4.1 Route quotes

Each quote carries a timestamp, TTL, source and data class, and an itemised all-in cost: amount at mid, FX spread, variable fee, fixed and intermediary fees. Unknown receiving-bank charges are excluded and labelled.

Route (fixture)	Known all-in (KZT)	Arrival (bd)	Result
R1 Bank wire	3,155,857	5–7	feasible, baseline
R2 Fintech	3,121,102	2–4	feasible
R3 Account	**3,099,163**	1–3	**selected**
R4 Cheap, slow	3,080,537	7–10	rejected: arrives after the safe date
R5 Stale quote	3,102,087	2–4	rejected: quote 24 h old

4.2 Deadline-constrained optimizer

- **Candidates:** 5 routes × 4 schedules (one transfer, two, three and four tranches) = 20 plans.
- **Hard constraints first:** latest safe arrival, quote freshness, route limits, cash reserve on every day of the liquidity timeline (rent, sponsor inflows), per-payment and daily caps. A plan that fails any of them is removed, however cheap it is.
- **Then cost:** surviving plans are ranked by known all-in cost, with stable tie-breaking.
- **INFEASIBLE:** if nothing passes, the optimizer returns INFEASIBLE with the reasons instead of a "least bad" plan.
- Every result has a decision record and a comparison with the R1 baseline on the same amount and deadline.

Fixture result: 45,000 CNY due 3 Sep, planned on 17 Aug; 9 of 20 plans pass; R3 in one transfer is selected at 3,099,163 KZT, **56,694 KZT (1.8%) less than the bank wire, simulated, not a guaranteed saving.** The lowest balance is 200,837 KZT above the 500,000 KZT reserve. If the sponsor is 10 days late, every plan is INFEASIBLE.

4.3 Stress and backtest

`stress.py` re-runs the plan under FX shocks, arrival delays, late inflows, route outages, stale quotes and changed beneficiaries (`evidence/stress.json`).

The v0.1 backtest (300 seeded rate paths, 45,000 CNY from KZT, 40-day deadline) shows: deadline met on **100%** of paths; timing-risk standard deviation **1.84% vs 4.46%** for one conversion at the deadline (**−59%**); mean saving **1.5%** vs a day-one bank wire. On a random walk timing has no edge on average; the saving comes from routing and fees, and splitting the purchase lowers risk. We publish the full distribution, including losing paths.

5. Tuition Shield

Beneficiaries are immutable versions. A new invoice is compared field by field (payee, account, bank, country, currency, reference) with the version the student confirmed through a trusted channel. Any difference blocks

payment (POL-010). A new version is held for 48 hours (POL-011) and can only be confirmed through a contact taken from the official website, never from the invoice itself. Instructions found inside invoice text are treated as data (POL-018).

6. Policy, approval and execution

- **Tiers:** L0 read-only, L1 advise, L2 confirm every movement, L3 autonomous up to a cap set by the student.
- **Policy engine:** 23 rules (POL-001…POL-023), verdicts ALLOW / ASK / DENY / RETURN_EXISTING, each with a rule ID, reason code and policy version. Missing critical state means DENY (fail closed). The agent cannot change policy (POL-023). Full catalogue: Security Self-Assessment.
- **Exact approval:** an intent has a SHA-256 fingerprint over amount, currencies, payee version, route, schedule and quote. The approval token is one-time, expires after 15 minutes, is stored only as a hash and is bound to the fingerprint. Any change after approval (for example +1,000 KZT) voids it (POL-021). Policy is re-checked at execution (POL-013).
- **Caps:** reservations are atomic; in a test, 20 parallel requests against a 10,000,000 KZT cap give exactly 3 successes.
- **Executor:** idempotency key per intent; a timeout followed by two retries gives one execution and one simulated debit. The kill switch belongs to the user only.

7. Payment lifecycle and reconciliation

`tracking.py` separates *submitted* from *paid*:

DRAFT → READY_FOR_APPROVAL → APPROVED → SUBMITTED_SIMULATED → PROCESSING_SIMULATED → SENT_SIMULATED → CREDIT_PENDING → CREDITED_SIMULATED

Only a credit that matches amount, payee, reference and date window closes the obligation. Partial or ambiguous matches go to RECONCILIATION_REQUIRED and need a manual resolution with an actor and evidence. Reusing a provider idempotency key with different content raises IDEMPOTENCY_CONFLICT.

8. Ledger, Evidence Pack, Evaluation Lab

- **Ledger:** every event is chained, `H_n = SHA-256(canonical_json(event) || H_(n−1))`. Personal fields are redacted; JSONL and CSV export; checkpoints let tail deletion be detected. If the ledger is unavailable, payments are denied (POL-019). The flagship run has 18 events.
- **Evidence Pack:** a manifest of the decision, profile versions, sources, quotes, approval fingerprint, lifecycle and ledger head, with file hashes. `verify_pack` returns VALID, INVALID (any byte changed) or INCOMPLETE. Export is blocked if a raw token would leak.
- **Evaluation Lab:** 16 cases from the spec (unsupported corridor, stale rule, scope mismatch, missing document, changed account, injection, cap, approval change, idempotency conflict and others), each with a time limit. An infrastructure error is never counted as a pass. The release gate passes only with 0 FAIL, 0 INFRA_ERROR and 0 critical false allows. Current run: **16/16 PASS, gate PASS** (`evidence/evaluation_run.json`). The gate covers this suite only; it is not proof of absolute security.

9. Interfaces

Tools for the model (`treasury/tools.py`, max 6 calls per turn, user-scoped, tokens never returned): `read_obligations`, `read_balances`, `parse_invoice`, `quote_routes`, `simulate_fx_plan`, `propose_payment_plan`, `request_approval`, `execute_sandbox_payment`, `read_audit_events`.

CLI:

```
python -m pytest -q                                    # 104 tests
python -m tuitionpilot.treasury flagship --out evidence   # 10-step flagship run
python -m tuitionpilot.treasury china --out evidence      # corridors, lifecycle, evidence pack, evaluation
python -m tuitionpilot.treasury replay --seed 7           # identical hashes expected
python -m tuitionpilot demo --out logs                    # v0.1 scenarios S1–S8
python tools/build_demo.py                                # rebuild the website from engine runs
```

Website (`demo/index.html`, deployable to Vercel with `vercel.json`, see `docs/DEPLOY.md`):

Page	What it does
Plan my payment	Bill, balance, cushion, rent and sponsor dates in → 20 plans, hard limits, cheapest safe plan, money-by-date chart, exact approval with fingerprint, lifecycle
Check my payment	Payer, direction, documents in → eligible methods and the last safe start date
Aigerim's story	8 interactive chapters with mini-games driven by recorded engine runs
Student cases	Aigerim (route vs deadline), Mira (changed bank details), Li (parent pays from China)
China corridors, Safety, Results, Team	Profiles and sources, policy rules, evaluation and backtest

The browser tools are ports of the Python code, not mock-ups. Parity tests: the eligibility checker matches `country.py` on **7,776** input combinations and the planner matches `optimizer.py` on **1,500 scenarios / 30,000 plans**, both with **0 differences** (`evidence/js_parity.json`, `evidence/planner_parity.json`).

10. Testing

104 automated tests (`pytest`): 39 for the v0.1 engine, 35 for the 2.0 core (mapped to QA-001…QA-026), 30 for the 2.1 layer. They cover rounding and rate direction, calendar and cut-offs, optimizer constraints and INFEASIBLE, beneficiary diff and hold, every policy rule path, approval replay, expiry and fingerprint change, concurrent caps, idempotency, kill switch, ledger tampering, corridor resolution, every eligibility status, profile freezing, lifecycle transitions, reconciliation, evidence pack verification, the evaluation gate and the planner API. Output: `evidence/run_2026-10-19/pytest_output.txt`, JUnit XML in `evidence/pytest-junit.xml`.

11. From sandbox to production

Sandbox component	Production replacement
Demo profiles	Profiles verified with each university's finance office, with source snapshots and review dates
Sandbox provider	Licensed partner APIs (bank, payment institution, education payment platform) behind the same interface
Fixture quotes	Partner quote API with locked validity windows
Typed approval code	Device-bound passkey (WebAuthn) signing the fingerprint
In-memory state and ledger	PostgreSQL with unique constraints; append-only storage with external anchoring of checkpoints
No identity layer	Authenticated users; policy changed only through a signed settings flow

Sandbox component	Production replacement
Static country list	Partner KYC/AML and sanctions screening that the agent cannot override

12. Team

- **Ekaterina Kulakova:** product, policy and security design, evaluation, website, presentation.
- **Dinara Khasyanova:** research on student payment problems and China corridors, scenarios, testing and documentation review.